

1. Find the Second Largest Element

Description

Given an array of integers, find the second largest distinct element in the array.

Input Format

First line contains an integer N.

Second line contains N space-separated integers.

Output Format

Print the second largest distinct element.

Sample Input

6
12 35 1 10 34 35

Sample Output

34

Explanation

The largest element is 35 and the second largest distinct element is 34.

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<2;i++){
        scanf("%d",&arr[i]);
    }
    int fm=0,sm=0;
    if(arr[0]>arr[1]){
        fm=arr[0];sm=arr[1];
    }
    else{
        fm=arr[1];sm=arr[0];
    }
    for(int i=2;i<n;i++){
        scanf("%d",&arr[i]);
        if(arr[i]>fm){
            sm=fm;fm=arr[i];
        }
        else if(arr[i]<fm && arr[i]>sm){
            sm=arr[i];
        }
    }
    printf("%d",sm);
    return 0;
}
```

2. Move All Zeros to the End

Description

Move all zero elements to the end while maintaining the order of non-zero elements.

Input Format

N

N space-separated integers

Output Format

Print the modified array.

Sample Input

7

0 1 0 3 12 0 5

Sample Output

1 3 12 5 0 0 0

Explanation

All non-zero elements retain their order, and zeros are moved to the end.

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d",&n);
    int arr[n];
    int j=0;
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
        if(arr[i]!=0){
            arr[j]=arr[i];
            j++;
        }
    }
    for(int i=j;i<n;i++){
        arr[i]=0;
    }
    for(int i=0;i<n;i++){
        printf("%d",arr[i]);
    }
    return 0;
}
```

3. Check if Array is Sorted

Description

Determine whether the array is sorted in ascending order.

Input Format

N

N space-separated integers

Output Format

Print YES if sorted, otherwise NO.

Sample Input

5

2 4 6 8 10

Sample Output

YES

Explanation

Every element is greater than or equal to the previous element.

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    for(int i=0;i<n-1;i++){
        if(arr[i]>arr[i+1]){
            printf("False");
            return 0;
        }
    }
    printf("True");
    return 0;
}
```

4. Find Missing Number

Description

An array contains numbers from 1 to N with one number missing. Find the missing number.

Input Format

N

N-1 space-separated integers

Sample Input

5

1 2 3 5

Sample Output

4

Explanation

Numbers from 1 to 5 should be present. Number 4 is missing.

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d",&n);
    int arr[n-1];
    for(int i=0;i<n-1;i++){
        scanf("%d",&arr[i]);
    }
    int a=arr[0];
    for(int i=0;i<n-2;i++){
        a+=1;
        if(a!=arr[i+1]){
            printf("%d",a);
            return 0;
        }
    }
    return 0;
}
```

5. Rotate Array by K Positions

Description

Rotate the array to the right by K positions.

Input Format

N

N space-separated integers

K

Sample Input

5

1 2 3 4 5

2

Sample Output

4 5 1 2 3

Explanation

After rotating twice to the right, the last two elements move to the front.

```
#include <stdio.h>
int main()
{
    int n,k;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    scanf("%d",&k);
    int m=0;
    int arr1[n];
    for(int i=k+1;i<n;i++){
        arr1[m]=arr[i];
        m++;
    }
    for(int i=0;i<=k;i++){
        arr1[m]=arr[i];
        m++;
    }
    for(int i=0;i<n;i++){
        printf("%d",arr1[i]);
    }
    return 0;
}
```

6. Remove Duplicates from a Sorted Array

Description

Remove duplicate elements from a sorted array.

Input Format

N

N space-separated integers

Sample Input

8

1 1 2 2 3 4 4 5

Sample Output

1 2 3 4 5

Explanation

Duplicate values are removed, leaving only unique elements.

```
#include <stdio.h>
int main()
{
    int n,k;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    for(int i=0;i<n-1;i++){
        if(arr[i]==arr[i+1]){
            arr[i]==-1000;
        }
        else{
            printf("%d",arr[i]);
        }
    }
    printf("%d",arr[n-1]);
    return 0;
}
```

7. Find Leaders in an Array

Description

An element is a leader if it is greater than all elements to its right.

Input Format

N

N space-separated integers

Sample Input

6

16 17 4 3 5 2

Sample Output

17 5 2

Explanation

17 is greater than all elements to its right.

5 is greater than 2.

2 is always a leader because it is the last element.

8. Find Frequency of Each Element

Description

Print the frequency of each distinct element.

Input Format

N

N space-separated integers

Sample Input

7

1 2 2 3 1 4 2

Sample Output

1 -> 2

2 -> 3

3 -> 1

4 -> 1

Explanation

Count the occurrences of each element.

1122234

```
#include <stdio.h>
#include <stdlib.h>
int comp(const void *a, const void *b){
    return (*(int*)a-*(int*)b);
}
int main()
{
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    qsort(arr,n,sizeof(arr[0]),comp);
    int count=1;
    for(int i=0;i<n;i++){
        if(arr[i]==arr[i+1])count++;
        else
        {printf("%d -> %d\n",arr[i],count);
        count=1;
        }
    }
    return 0;
}
```


9. Find Pair with Given Sum

Description

Check whether any pair of elements adds up to a given target.

Input Format

N

N space-separated integers

Target

Sample Input

5

2 7 11 15 3

9

Sample Output

YES

Explanation

2 + 7 = 9.

```
#include <stdio.h>
#include <stdlib.h>
int comp(const void *a, const void *b){
    return (*(int*)a-*(int*)b);
}
int main()
{
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    int target;
    scanf("%d",&target);
    qsort(arr,n,sizeof(arr[0]),comp);
    int i=arr[0],j=arr[n-1];
    while(i<j){
        if(i+j==target)
            {printf("Yes");return 0;}
        else if(i+j>target)j--;
        else i++;
    }
    printf("False");
    return 0;
}
```

10. Maximum Difference

Description

Find the maximum difference between two elements where the larger element comes after the smaller one.

Sample Input

7

2 3 10 6 4 8 1

Sample Output

8

Explanation

Maximum difference is $10 - 2 = 8$.

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    int min=arr[0],diff=0;
    for(int i=1;i<n;i++){
        if(arr[i]-min>diff){
            diff=arr[i]-min;
        }
        if(arr[i]<min){
            min=arr[i];
        }
    }
    printf("%d",diff);
    return 0;
}
```

11. Product of Array Except Self

Description

For every index, find the product of all elements except itself.

Sample Input

4

1 2 3 4

Sample Output

24 12 8 6

Explanation

For index 0: $2 \times 3 \times 4 = 24$

For index 1: $1 \times 3 \times 4 = 12$

and so on.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,count=0,prod=1;
```

```
    scanf("%d",&n);
```

```
    int arr[n];
```

```
    for(int i=0;i<n;i++){
```

```
        scanf("%d",&arr[i]);
```

```
    }
```

```
    for(int i=0;i<n;i++){
```

```
        if(arr[i]!=0){
```

```
            prod*=arr[i];
```

```
        }
```

```
        else{
```

```
            count++;
```

```
        }
```

```
    }
```

```
    for(int i=0;i<n;i++){
```

```
        if(count==0){
```

```
            printf("%d ",prod/arr[i]);
```

```
        }
```

```
        else if(count>1)printf("0 ");
```

```
        else{
```

```
            if(arr[i]==0){
```

```
                printf("%d ",prod);
```

```
            }
```

```
            else{
```

```
                printf("0 ");
```

```
            }
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

12. Longest Consecutive Sequence

Description

Find the length of the longest consecutive sequence.

Sample Input

6

100 4 200 1 3 2

Sample Output

4

Explanation

Sequence 1, 2, 3, 4 has length 4.

```
#include <stdio.h>
#include <stdlib.h>
int comp(const void *a,const void *b){
    return (*(int *)a-*(int*)b);
}
int main()
{
    int n,max=0,count=1;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    qsort(arr,n,sizeof(arr[0]),comp);
    for(int i=1;i<n;i++){
        if(arr[i-1]+1==arr[i]){
            count++;
        }
        else{
            if(max<count)max=count;
            count=1;
        }
    }
    printf("%d",max);
    return 0;
}
```

13. Equilibrium Index

Description

Find an index where left sum equals right sum.

Sample Input

5

1 3 5 2 2

Sample Output

2

Explanation

Left sum = $1 + 3 = 4$

Right sum = $2 + 2 = 4$

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d",&n);
    int arr[num];
    int sum=0;
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
        sum+=arr[i];
    }
    int left=arr[0];
    for(int i=1;i<n;i++){
        if(sum-left-arr[i]==left){
            printf("%d",i);
            return 0;
        }else{
            left+=arr[i];
        }
    }
    printf("-1");
    return 0;
}
```

14. Rearrange Positive and Negative Numbers

Description

Arrange positive and negative numbers alternately.

Sample Input

6

1 -2 3 -4 5 -6

Sample Output

1 -2 3 -4 5 -6

Explanation

The array is already in alternating order.

```
#include <stdio.h>
#include <stdlib.h>
int comp(const void *a,const void *b){
    return (*(int*)a-*(int*)b);
}
int main()
{
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    qsort(arr,n,sizeof(arr[0]),comp);
    int count=1;
    for(int i=1;i<n;i++){
        if(arr[i-1]==arr[i]){
            count++;
        }
        if(count>n/2){
            printf("%d",arr[i-1]);
            return 0;
        }
    }
    printf("-1");
    return 0;
}
```

15. Majority Element

Description

Find the element appearing more than $N/2$ times.

Sample Input

7

2 2 1 2 3 2 2

Sample Output

2

Explanation

2 appears 5 times, which is more than $7/2$.

```
#include <stdio.h>
#include <stdlib.h>
int comp(const void *a,const void *b){
    return (*(int*)a-*(int*)b);
}
int main()
{
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    qsort(arr,n,sizeof(arr[0]),comp);
    int count=1;
    for(int i=1;i<n;i++){
        if(arr[i-1]==arr[i]){
            count++;
        }
        if(count>n/2){
            printf("%d",arr[i-1]);
            return 0;
        }
    }
    printf("-1");
    return 0;
}
```

16. Maximum Sum Subarray

Description

Find the contiguous subarray with the maximum sum.

Sample Input

9

-2 1 -3 4 -1 2 1 -5 4

Sample Output

6

Explanation

Subarray [4, -1, 2, 1] has maximum sum 6.

```
#include <stdio.h>
```

```
int main{
```

```
int sum=0;
```

```
int n;
```

```
int nums[n];
```

```
for(int i=0;i<n;i++){
```

```
scanf("%d",&nums[i]);
```

```
}
```

```
    int maxa=nums[0];
```

```
    for(int i=0;i<n;i++){
```

```
        sum+=nums[i];
```

```
        if(sum>maxa)maxa=sum;
```

```
        if(sum<0){
```

```
            sum=0;
```

```
        }
```

```
    }
```

```
printf("%d",maxa);
```

```
return 0;
```

```
}
```


17. Trapping Rain Water

Description

Calculate total water trapped between bars.

Sample Input

12

0 1 0 2 1 0 1 3 2 1 2 1

Sample Output

6

Explanation

Total trapped water equals 6 units.

```
#include <stdio.h>
int main()
{
    int num;
    scanf("%d",&num);
    int arr[num];
    for(int i=0;i<num;i++){
        scanf("%d",&arr[i]);
    }
    int ref;
    int maxa=arr[0];
    for(int k=0;k<num;k++){
        if(arr[k]>maxa){
            maxa=arr[k];
            ref=k;
        }
    }
    int sum=0;
    int i=0;
    for(int j=1;j<ref;j++){
        if(arr[i]-arr[j]>0){
            sum+=(arr[i]-arr[j]);
        }else{
            i=j;
        }
    }
    i=num-1;
    for(int j=num-1;j>ref;j--){
        if(arr[i]-arr[j]>0){
            sum+=(arr[i]-arr[j]);
        }else{
            i=j;
        }
    }
    printf("%d",sum);
}
```

18. Container With Most Water

Description

Find two lines that can contain the maximum water.

Sample Input

9

1 8 6 2 5 4 8 3 7

Sample Output

49

Explanation

Lines at indices 1 and 8 hold:

$$\min(8,7) \times (8-1) = 49$$

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    scanf("%d",&n);
```

```
    int arr[n];
```

```
    for(int i=0;i<n;i++){
```

```
        scanf("%d",&arr[i]);
```

```
    }
```

```
    int left=0,right=n-1 ;
```

```
    int max=0,min=1;
```

```
    while(left<right){
```

```
        if(arr[left]<arr[right])min=arr[left];
```

```
        else min=arr[right];
```

```
        min*=(right-left);
```

```
        if(min>max)max=min;
```

```
        if(arr[left]<arr[right])left++;
```

```
        else right--;
```

```
    }
```

```
    printf("%d",max);
```

```
    return 0;
```

```
}
```

19. Merge Overlapping Intervals

Description

Merge all overlapping intervals.

Sample Input

4

1 3

2 6

8 10

15 18

Sample Output

1 6

8 10

15 18

Explanation

Intervals [1,3] and [2,6] overlap and are merged.

20. Maximum Product Subarray

Description

Find the contiguous subarray having the maximum product.

Sample Input

4

2 3 -2 4

Sample Output

6

Explanation

Subarray [2,3] gives the maximum product 6.

21. Find the Union of Two Arrays

Description

Given two arrays, find all distinct elements present in either array.

Input Format

First line: N

Second line: N integers

Third line: M

Fourth line: M integers

Sample Input

5

1 2 3 4 5

4

1 2 6 7

Sample Output

1 2 3 4 5 6 7

Explanation

Union contains all unique elements from both arrays.

```
#include <stdio.h>
```

```
int main() {  
    int n, m, a[1000], x;
```

```
    scanf("%d",&n);
```

```
    for(int i=0;i<n;i++){  
        scanf("%d",&a[i]);
```

```
    scanf("%d",&m);
```

```
    for(int i=0;i<m;i++){  
        scanf("%d",&x);
```

```
        int found=0;
```

```
        for(int j=0;
```

```
        j < n; j++) {  
            if (a[j] == x) {  
                found = 1;  
                break;  
            }  
        }
```

```
        if (!found)  
            a[n++] = x;
```

```
    }
```

```
    for (int i = 0; i < n; i++) {  
        int first = 1;
```

```
        for (int j = 0; j < i; j++) {  
            if (a[i] == a[j]) {  
                first = 0;
```

```

        break;
    }
}

if (first) {
    printf("%d", a[i]);

    int more = 0;
    for (int k = i + 1; k < n; k++) {
        int unique = 1;
        for (int j = 0; j < k; j++) {
            if (a[k] == a[j]) {
                unique = 0;
                break;
            }
        }
        if (unique) {
            more = 1;
            break;
        }
    }

    if (more)
        printf(" ");
}

return 0;
}

```

22. Find the Intersection of Two Arrays

Description

Find common elements present in both arrays.

Sample Input

5
1 2 3 4 5
5
3 4 5 6 7

Sample Output

3 4 5

Explanation

3, 4, and 5 are present in both arrays.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    scanf("%d",&n);
```

```
    int arr[n];
```

```
    for(int i=0;i<n;i++){
```

```
        scanf("%d",&arr[i]);
```

```
    }
```

```
    int m;
```

```
    scanf("%d",&m);
```

```
    int arr1[m];
```

```
    for(int i=0;i<m;i++){
```

```
        scanf("%d",&arr1[i]);
```

```
    }
```

```
    for(int i=0;i<n;i++){
```

```
        for(int j=0;j<m;j++){
```

```
            if(arr[i]==arr1[j]){
```

```
                printf("%d ",arr[i]);
```

```
            }
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

23. Find the First Repeating Element

Description

Find the first element that appears more than once.

Sample Input

7

10 5 3 4 3 5 6

Sample Output

5

Explanation

5 is the first element whose repetition occurs earliest.

```
#include <stdio.h>
#include <limits.h>
int main()
{   int n;
    scanf("%d",&n);
    int arr[n];
    int count[100000]={0};
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
        // count[i]=0;
    }
    for(int i=0;i<n;i++){
        count[arr[i]]++;
    }
    for(int i=0;i<n;i++){
        if(count[arr[i]]>1){printf("%d",arr[i]);return 0;}
    }

    return 0;
}
```


24. Find the First Non-Repeating Element

Description

Find the first element that occurs exactly once.

Sample Input

6

9 4 9 6 7 4

Sample Output

6

Explanation

6 is the first element with frequency 1.

```
#include <stdio.h>
#include <limits.h>
int main()
{   int n;
    scanf("%d",&n);
    int arr[n];
    int count[100000]={0};
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
        // count[i]=0;
    }
    for(int i=0;i<n;i++){
        count[arr[i]]++;
    }
    for(int i=0;i<n;i++){
        if(count[arr[i]]==1){printf("%d",arr[i]);return 0;}
    }
    return 0;
}
```

25. Maximum Consecutive Ones

Description

Find the maximum number of consecutive 1s in a binary array.

Sample Input

8

1 1 0 1 1 1 0 1

Sample Output

3

Explanation

The longest sequence of consecutive 1s has length 3.

```
#include <stdio.h>
int main()
{
    int n,count=0;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    int maxi=0;
    for(int i=0;i<n;i++){
        if(arr[i]==1){
            count++;
            if(count>maxi)maxi=count;
        }
        else{
            count=0;
        }
    }
    printf("%d",maxi);
    return 0;
}
```

26. Find All Duplicate Elements

Description

Print all elements that appear more than once.

Sample Input

8

1 2 3 1 2 4 5 2

Sample Output

1 2

Explanation

Elements 1 and 2 occur multiple times.

```
#include <stdio.h>
```

```
int main() {  
    int n, x;  
    int freq[1001] = {0};
```

```
    scanf("%d",&n);
```

```
    for(int i=0;i<n;i++){  
        scanf("%d",&x);  
        freq[x]++;  
    }
```

```
    int first=1;
```

```
    for (int i=0;i<=1000;i++){  
        if (freq[i]>1){  
            if(!first)printf(" ");  
            printf("%d",i);  
            first = 0;  
        }  
    }
```

```
    return 0;
```

```
}
```

27. Largest Sum Contiguous Subarray of Size K

Description

Find the maximum sum among all subarrays of size K.

Sample Input

6
2 1 5 1 3 2
3

Sample Output

9

Explanation

Subarray [5,1,3] has sum 9.

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    int k;
    scanf("%d",&k);
    int left=0,sum=0,maxi=0;
    for(int right=0;right<n;right++){
        sum+=arr[right];
        if(right-left+1>=k){
            if(sum>maxi)maxi=sum;
            sum=sum-arr[left];
            left++;
        }
    }
    printf("%d",maxi);
    return 0;
}
```

28. Find the Smallest Positive Missing Number

Description

Find the smallest positive integer missing from the array.

Sample Input

5

3 4 -1 1 2

Sample Output

5

Explanation

Numbers 1,2,3,4 exist, so 5 is missing.

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, a[1000], freq[1001] = {0};
```

```
    scanf("%d",&n);
```

```
    for (int i=0;i<n;i++) {
```

```
        scanf("%d",&a[i]);
```

```
        if(a[i]>0 && a[i]<=1000)
```

```
            freq[a[i]]=1;
```

```
    }
```

```
    for(int i=1;i<=1001;i++) {
```

```
        if (freq[i]==0){
```

```
            printf("%d",i);
```

```
            break;
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

29. Count Inversions in an Array

Description

An inversion is a pair (i,j) such that $i < j$ and $arr[i] > arr[j]$.

Sample Input

5

2 4 1 3 5

Sample Output

3

Explanation

Inversions are (2,1), (4,1), and (4,3).

```
#include <stdio.h>
```

```
int main() {  
    int n, a[1000];  
    int count=0;  
  
    scanf("%d",&n);  
  
    for (int i=0;i<n;i++)  
        scanf("%d",&a[i]);  
  
    for(int i=0;i<n;i++){  
        for (int j=i+1;j<n;j++) {  
            if(a[i]>a[j])  
                count++;  
        }  
    }  
  
    printf("%d",count);  
  
    return 0;  
}
```

30. Find Peak Element

Description

Find an element greater than its adjacent elements.

Sample Input

6

1 3 20 4 1 0

Sample Output

20

Explanation

20 is greater than both 3 and 4.

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, a[1000];
```

```
    scanf("%d", &n);
```

```
    for(int i=0;i<n;i++)
```

```
        scanf("%d",&a[i]);
```

```
    for(int i=1;i<n-1;i++) {
```

```
        if(a[i]>a[i-1] && a[i]>a[i+1]) {
```

```
            printf("%d",a[i]);
```

```
            break;
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

31. Maximum Circular Subarray Sum

Description

Find the maximum subarray sum considering the array as circular.

Sample Input

5

8 -8 9 -9 10

Sample Output

19

Explanation

Circular subarray [10, 8, -8, 9] gives sum 19.

32. Sliding Window Maximum

Description

Find the maximum element in every window of size K.

Sample Input

8

1 3 -1 -3 5 3 6 7

3

Sample Output

3 3 5 5 6 7

Explanation

Each value is the maximum of a window of size 3.

33. Minimum Number of Jumps to Reach End Description

Each element represents the maximum jump length from that position.

Sample Input

11

1 3 5 8 9 2 6 7 6 8 9

Sample Output

3

Explanation

Minimum jumps required to reach the last index are 3.

34. Find Subarray with Given Sum

Description

Find the starting and ending indices of a subarray whose sum equals K.

Sample Input

5

1 4 20 3 10

33

Sample Output

2 4

Explanation

$20 + 3 + 10 = 33$.

35. Chocolate Distribution Problem

Description

Distribute packets such that the difference between maximum and minimum chocolates is minimized.

Sample Input

7

7 3 2 4 9 12 56

3

Sample Output

2

Explanation

Choosing packets [2,3,4] gives difference 2.

36. Find the Median of Two Sorted Arrays

Description

Find the median of two sorted arrays.

Sample Input

3
1 3 5
3
2 4 6

Sample Output

3.5

Explanation

Merged array = [1,2,3,4,5,6].

Median = $(3+4)/2 = 3.5$.

```
#include <stdio.h>
```

```
int main() {  
    int n, m;  
    int a[1000], b[1000], c[2000];  
  
    scanf("%d",&n);  
    for (int i=0;i<n;i++)  
        scanf("%d",&a[i]);  
  
    scanf("%d",&m);  
    for (int i=0;i<m;i++)  
        scanf("%d",&b[i]);  
  
    int i=0, j=0, k=0;  
  
    while(i<n && j<m) {  
        if(a[i]<b[j])  
            c[k++]=a[i++];  
        else  
            c[k++]=b[j++];  
    }  
  
    while(i<n)  
        c[k++]=a[i++];  
  
    while(j<m)  
        c[k++]=b[j++];  
  
    if (k%2==0)  
        printf("%.1f", (c[k/2-1]+c[k/2])/2.0);  
    else  
        printf("%d", c[k/2]);  
  
    return 0;  
}
```

37. Maximum Length Bitonic Subarray

Description

Find the longest subarray that first increases and then decreases.

Sample Input

8

12 4 78 90 45 23 10 5

Sample Output

7

Explanation

Bitonic subarray is [4,78,90,45,23,10,5].

38. Rainwater Harvesting Between Buildings

Description

Calculate trapped rainwater between buildings.

Sample Input

6

3 0 0 2 0 4

Sample Output

10

Explanation

Total trapped water is 10 units

39. Find the K-th Largest Element

Description

Find the K-th largest element in the array.

Sample Input

6

7 10 4 3 20 15

3

Sample Output

10

Explanation

Sorted descending: 20,15,10,7,4,3.

3rd largest element is 10.

```
#include <stdio.h>
```

```
int main() {  
    int n, k, a[1000];  
  
    scanf("%d",&n);  
  
    for (int i=0;i<n;i++)  
        scanf("%d",&a[i]);  
  
    scanf("%d",&k);  
  
    for(int i=0;i<n-1;i++) {  
        for(int j=0;j<n-i-1;j++) {  
            if(a[j]<a[j+1]){  
                int temp=a[j];  
                a[j]=a[j+1];  
                a[j+1]=temp;  
            }  
        }  
    }  
  
    printf("%d",a[k-1]);  
  
    return 0;  
}
```


40. Maximum Rectangle Area in Histogram

Description

Given bar heights of a histogram, find the largest rectangular area.

Sample Input

7

6 2 5 4 5 1 6

Sample Output

12

Explanation

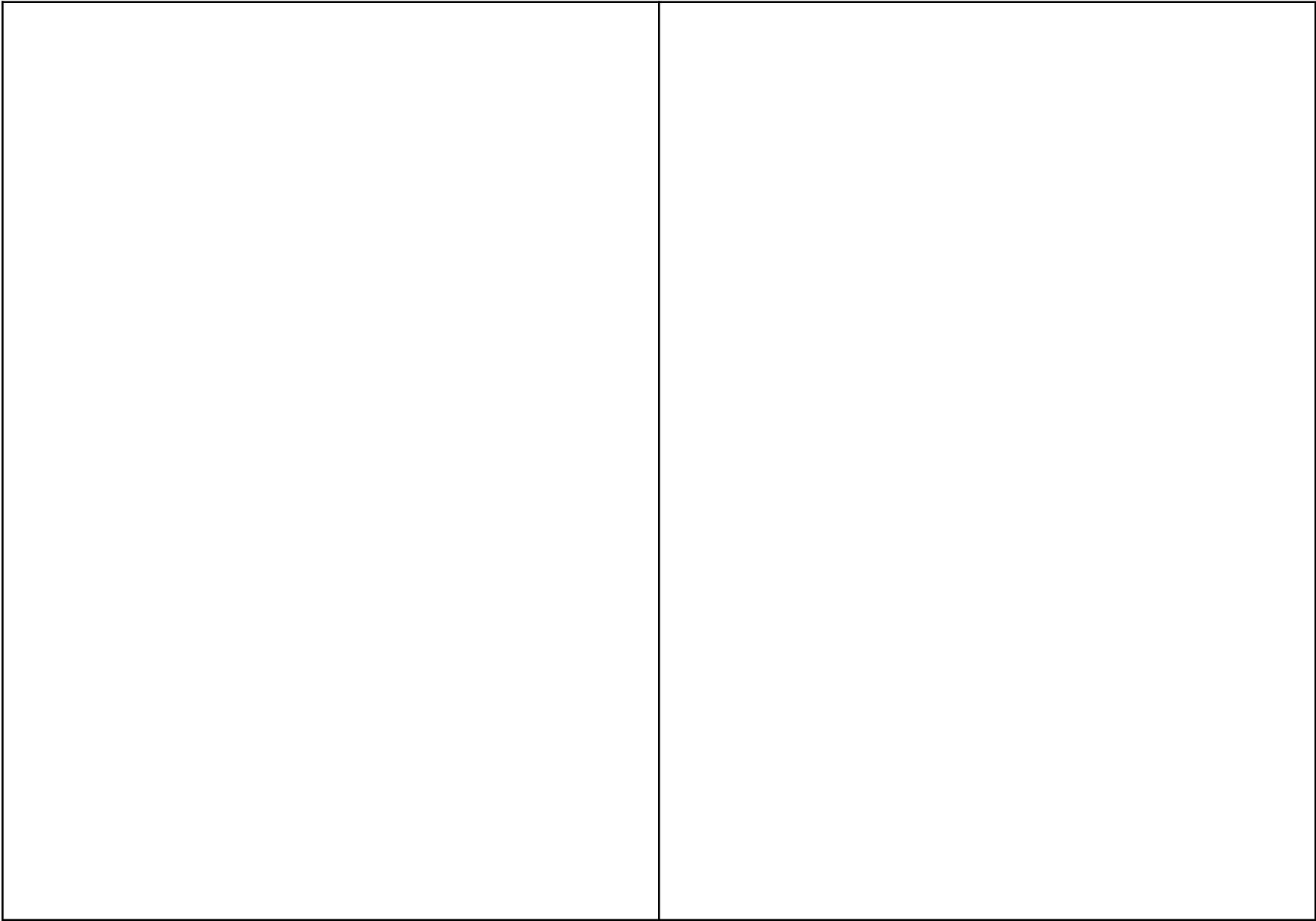
The maximum rectangle area is formed by bars with heights 4,5,5 resulting in area 12.

```
#include <stdio.h>
```

```
int main() {  
    int n, a[1000];  
    int maxArea = 0;  
  
    scanf("%d",&n);  
  
    for (int i=0;i<n;i++)  
        scanf("%d",&a[i]);  
  
    for (int i=0;i<n;i++) {  
        int minHeight=a[i];  
  
        for(int j=i;j<n;j++){  
            if(a[j]<minHeight)  
                minHeight=a[j];  
  
            int area=minHeight*(j-i+1);  
  
            if (area>maxArea)  
                maxArea=area;  
        }  
    }  
  
    printf("%d",maxArea);  
  
    return 0;  
}
```

--	--

--	--



--	--

--	--

--	--

